

GET vs POST Methods



You said:

get and post

ChatGPT said:

In **Express.js**, **GET** and **POST** are HTTP methods used to send and receive data between client (browser) and server.

◆ 1. GET Method

👉 Used to **request (fetch) data** from the server

✅ Features:

- Data is sent in **URL**
- Visible in browser
- Limited data size

- Used for **reading data**

Example:

```
JavaScript
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("This is GET request");
});

app.listen(3000);
```

URL Example:

```
http://localhost:3000/?name=Aamir
```

Access data:

```
JavaScript
req.query.name
```

2. POST Method

 Used to **send data to server** (like form submission)

Features:

- Data is sent in **body**
- Not visible in URL
- No size limit (large data)

- Used for **creating/updating data**

✦ Example:

```
JavaScript
const express = require("express");
const app = express();

app.use(express.json()); // for JSON data

app.post("/data", (req, res) => {
  console.log(req.body);
  res.send("This is POST request");
});

app.listen(3000);
```

🔥 Difference (Interview Important)

Feature	GET	POST
Purpose	Fetch data	Send data
Data location	URL (query)	Body
Security	Less secure	More secure
Size	Limited	Large
Use case	Read data	Create data

💡 Real-Life Example

- **GET** → Opening Google search page
- **POST** → Submitting login form (username/password)

🎯 Interview Line (Short Answer)

👉 "GET is used to retrieve data from server, while POST is used to send data to server securely in request body."

If you want, I can give:

- ✓ Form handling (HTML + Express)
- ✓ CRUD API (GET, POST, PUT, DELETE full project)
- ✓ Interview questions on HTTP methods

You said:

get and post all concept

ChatGPT said:

Alright — let's go **deep + interview-ready** on **GET and POST** in **Express.js** and HTTP.

1. What are GET and POST?

They are **HTTP methods** used in **client-server communication**.

- **GET** → Read / Fetch data
- **POST** → Send / Create data

They are part of **REST API design**.

2. GET Method (Complete Concept)

Purpose:

Retrieve data from server

How it works:

- Data is sent via **URL (query parameters)**
- Server returns response

Example URL:

```
/search?name=aamir&age=22
```

Express Example:

```
JavaScript
app.get("/user", (req, res) => {
  const name = req.query.name;
  res.send(`Hello ${name}`);
});
```


Key Points:

- ✓ Safe (does not change data)
- ✓ Idempotent (same request = same result)
- ✓ Cached by browser
- ✓ Bookmark possible
- ✗ Data visible in URL
- ✗ Limited length (~2048 chars)

3. POST Method (Complete Concept)

Purpose:

Send data to server (create/update)

Express Example:

```
JavaScript
app.use(express.json());

app.post("/user", (req, res) => {
  const data = req.body;
  res.send(`User ${data.name} created`);
});
```

Key Points:

- ✓ Data sent in **request body**
- ✓ More secure than GET
- ✓ No size limit (large data allowed)
- ✓ Used for forms, login, file upload
- ✗ Not cached

- ❌ Not bookmarkable

🔪 4. GET vs POST (Deep Comparison)

Feature	GET 🟢	POST 🔴
Purpose	Fetch data	Send/Create data
Data Location	URL (query params)	Body
Security	Low	Higher
Idempotent	Yes	No
Cache	Yes	No
Bookmark	Yes	No
Data Size	Limited	Unlimited

🔒 5. Security Concept (Important)

👉 Both GET and POST are **not fully secure alone**

- Use **HTTPS** for encryption
- Never send passwords in GET
- POST is better but still needs HTTPS

📦 6. Request Structure

GET Request:

```
GET /user?name=aamir HTTP/1.1
```

POST Request:

```
POST /user HTTP/1.1
Content-Type: application/json

{
  "name": "aamir"
}
```

7. Types of Data Sending

GET:

```
JavaScript  
req.query
```

POST:

```
JavaScript  
req.body
```

URL Params:

```
JavaScript  
app.get("/user/:id", (req, res) => {  
    res.send(req.params.id);  
});
```

8. Real-Life Use Cases

◆ GET:

- Fetch users list
- Search data
- Load webpage

◆ POST:

- Login form
- Signup form
- Upload file

- Add product

⚠️ 9. Common Mistakes (Interview Trap)

- ✗ Sending password in GET
- ✗ Using POST for fetching data
- ✗ Forgetting middleware (`express.json()`)
- ✗ Not validating request body

🧪 10. Example (Frontend + Backend)

HTML Form (POST):

```
HTML
<form action="/user" method="POST">
  <input name="name" />
  <button type="submit">Send</button>
</form>
```

Express:

```
JavaScript
app.use(express.urlencoded({ extended: true }));

app.post("/user", (req, res) => {
  res.send(req.body.name);
});
```

🚀 11. REST API Usage

Operation	Method
Get all data	GET
Get single	GET
Create	POST
Update	PUT
Delete	DELETE

12. Interview Answer (Best)



"GET is used to retrieve data and sends data in URL, while POST is used to send data in request body for creating or updating resources. GET is idempotent and cacheable, while POST is not."



Pro Tip (Very Important)

 Use:

- GET → when **no data change**
- POST → when **data changes in database**

If you want next level 

I can teach you:

- ✓ PUT vs PATCH vs DELETE
- ✓ Full CRUD API project
- ✓ Postman testing
- ✓ Real interview coding questions

You said:

handling post request all concept

ChatGPT said:

Let's go **deep into handling POST requests** in **Express.js** — this is **very important for interviews + real projects** 



1. What is Handling POST Request?

 It means **receiving data from client** → **processing it** → **sending response**

Used in:

- Login forms
- Signup
- APIs

- File uploads

2. Basic Flow of POST Request

```
Client (form / frontend)
  ↓
POST request
  ↓
Server (Express)
  ↓
Process data (req.body)
  ↓
Send response (res.send / res.json)
```

3. Basic POST Handling

Code:

```
JavaScript
const express = require("express");
const app = express();

// Middleware (IMPORTANT)
app.use(express.json());

app.post("/user", (req, res) => {
  console.log(req.body);
  res.send("Data received");
});

app.listen(3000);
```

4. Middleware (Most Important Concept)

 Without middleware, `req.body` = **undefined**

Types:

◆ JSON Data:

```
JavaScript
app.use(express.json());
```

◆ Form Data:

```
JavaScript
app.use(express.urlencoded({ extended: true }));
```

5. Types of POST Data

1. JSON Data

```
JSON
{
  "name": "Aamir",
  "age": 22
}
```

👉 Access:

```
JavaScript
req.body.name
```

2. Form Data

```
HTML
<form method="POST" action="/user">
  <input name="name">
</form>
```

👉 Access:

```
JavaScript
req.body.name
```

3. URL Encoded

```
name=Aamir&age=22
```

4. File Upload (Advanced)

👉 Use **multer**

```
Bash
npm install multer
```

```
JavaScript
const multer = require("multer");
const upload = multer({ dest: "uploads/" });

app.post("/upload", upload.single("file"), (req, res) => {
  res.send("File uploaded");
});
```



6. Validation (Very Important)

👉 Never trust user input !

```
JavaScript
app.post("/user", (req, res) => {
  const { name, age } = req.body;

  if (!name || !age) {
    return res.status(400).send("All fields required");
  }

  res.send("Valid data");
});
```



7. Error Handling

```
JavaScript
app.post("/user", (req, res) => {
  try {
    const data = req.body;
    res.json(data);
  } catch (error) {
    res.status(500).send("Server error");
  }
});
```


8. Async/Await (Real Project)

```
JavaScript
app.post("/user", async (req, res) => {
  try {
    const user = req.body;
    // simulate DB save
    res.status(201).json(user);
  } catch (err) {
    res.status(500).send(err.message);
  }
});
```

9. Database Example (Real Use)

```
JavaScript
app.post("/user", async (req, res) => {
  const { name } = req.body;

  // Example DB save
  const newUser = { name };

  res.status(201).json({
    message: "User created",
    data: newUser
  });
});
```

10. Testing POST Request

◆ Using Postman (Best)

- Method: POST
- Body → JSON

- Send request

◆ Using Fetch (Frontend)

```
JavaScript
fetch("/user", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({ name: "Aamir" })
});
```

✂ 11. Common Mistakes ✖

- ✖ Forgetting `express.json()`
- ✖ Not validating data
- ✖ Sending wrong content-type
- ✖ Not handling errors
- ✖ Trusting user input blindly

🎯 12. Interview Questions

Q1: Why middleware needed?

👉 To parse incoming request body

Q2: What is `req.body`?

👉 Object containing client data

Q3: Difference between JSON & URL encoded?

👉 JSON → structured, URL encoded → key=value format

Q4: Status code for POST?

👉 201 (Created)



13. Pro Level Tips

- ✓ Always validate input
- ✓ Use status codes properly
- ✓ Use async/await
- ✓ Secure API (JWT/Auth)
- ✓ Use MVC structure



Final Summary



Handling POST =

Receive → Parse → Validate → Process → Respond

If you want next level 🔥

I can help you build:

- ✓ Full CRUD API project
- ✓ Login/Register system (JWT)
- ✓ Connect with MongoDB / MySQL
- ✓ Real backend interview preparation

You said:

revisiting js with oops

ChatGPT said:

Great decision — **revising JavaScript with OOP (Object-Oriented Programming)** will make your fundamentals very strong 🙌 and help in interviews + projects.

Let's go **step-by-step from basics → advanced** 🚀



1. What is OOP in JavaScript?



OOP = programming using **objects and classes**

Instead of writing code like:

```
JavaScript
let name = "Aamir";
let age = 22;
```

We use:

```
JavaScript
let user = {
  name: "Aamir",
  age: 22
};
```

2. Main Pillars of OOP

1. Encapsulation

👉 Data + methods together

2. Inheritance

👉 One class uses properties of another

3. Polymorphism

👉 Same method, different behavior

4. Abstraction

👉 Hide internal details

3. Objects in JavaScript

```
JavaScript
const student = {
  name: "Aamir",
  marks: 90,
  greet: function() {
    console.log("Hello");
  }
};

student.greet();
```



4. Constructor Function (Old Way)

```
JavaScript
function Person(name, age) {
    this.name = name;
    this.age = age;
}

const p1 = new Person("Aamir", 22);
```



5. Classes (Modern JavaScript)

```
JavaScript
class Person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }

    greet() {
        console.log("Hello " + this.name);
    }
}

const p1 = new Person("Aamir", 22);
p1.greet();
```



6. Inheritance

```
JavaScript
class Student extends Person {
    constructor(name, age, marks) {
        super(name, age);
        this.marks = marks;
    }

    study() {
        console.log(this.name + " is studying");
    }
}

const s1 = new Student("Aamir", 22, 95);
s1.study();
```

7. Polymorphism

```
JavaScript
class Animal {
  sound() {
    console.log("Animal makes sound");
  }
}

class Dog extends Animal {
  sound() {
    console.log("Dog barks");
  }
}

const d = new Dog();
d.sound(); // overridden
```

8. Encapsulation (Private Fields)

```
JavaScript
class BankAccount {
  #balance = 0;

  deposit(amount) {
    this.#balance += amount;
  }

  getBalance() {
    return this.#balance;
  }
}

const acc = new BankAccount();
acc.deposit(1000);
console.log(acc.getBalance());
```

9. Abstraction

 Hide complex logic

```
JavaScript
class Car {
  start() {
    this.#engineStart();
    console.log("Car started");
  }

  #engineStart() {
    console.log("Engine running...");
  }
}
```

10. Prototype (Important for Interview)

👉 JavaScript is **prototype-based**, not class-based internally

```
JavaScript
function Person(name) {
  this.name = name;
}

Person.prototype.sayHello = function() {
  console.log("Hello " + this.name);
};
```

11. this Keyword

👉 Refers to current object

```
JavaScript
class User {
  constructor(name) {
    this.name = name;
  }

  show() {
    console.log(this.name);
  }
}
```

12. Common Mistakes

- ✗ Forgetting **new** keyword
- ✗ Losing **this** context
- ✗ Not using **super()** in child class

✗ Confusing prototype vs class



13. Practice Questions

1. Create a class **Car** with properties and methods
2. Create **Employee** → **Manager** inheritance
3. Implement bank system with encapsulation
4. Override method in child class
5. Use prototype to add method



14. Interview Answer (Short)



"OOP in JavaScript is based on objects and prototypes. It includes encapsulation, inheritance, polymorphism, and abstraction. ES6 introduced classes which are syntactic sugar over prototypes."



Next Level (I recommend)

Since you're learning full stack:



Next learn:

- Closures
- Call, Apply, Bind
- Event Loop
- Async JS
- Design Patterns

If you want 🔥

I can give you:

- ✓ 30 OOP coding questions with solutions
- ✓ Real interview questions (Amazon, TCS, etc.)
- ✓ Mini project using OOP (Bank / E-commerce system)

You said:

object prototypes all concept

ChatGPT said:

Perfect 🔥 — **Object Prototypes** is one of the **most important + confusing** topics in JavaScript. Let's make it **crystal clear from basics → advanced (interview level)** 🚀

🧠 1. What is Prototype?

👉 In JavaScript, **every object has a hidden property called** `[[Prototype]]`

It is accessed using:

```
JavaScript
__proto__
```

👉 It allows **objects to inherit properties and methods**

🔗 2. Prototype Chain (Core Concept)

```
JavaScript
const obj = {};
```

👉 Behind the scenes:

```
JavaScript
obj.__proto__ === Object.prototype // true
```

👉 And:

```
JavaScript
Object.prototype.__proto__ === null
```

📁 Chain:

```
obj → Object.prototype → null
```

👉 This is called **Prototype Chain**

3. Why Prototype?

👉 To **share methods** instead of duplicating

❌ Bad:

```
JavaScript
function Person(name) {
  this.name = name;
  this.sayHello = function() {
    console.log("Hello " + this.name);
  };
}
```

👉 Every object creates new function (memory waste)

✅ Good (Using Prototype)

```
JavaScript
function Person(name) {
  this.name = name;
}

Person.prototype.sayHello = function() {
  console.log("Hello " + this.name);
};

const p1 = new Person("Aamir");
const p2 = new Person("Ali");

p1.sayHello();
```

👉 Same function shared 👍

4. How **new** Works Internally (Very Important)

```
JavaScript
const p1 = new Person("Aamir");
```

👉 Steps:

1. Create empty object `{}`
2. Set prototype → `obj.__proto__ = Person.prototype`

3. Bind **this** to object

4. Return object

5. **__proto__** vs **prototype**

Feature	__proto__	prototype
Belongs to	Object	Function
Purpose	Access prototype	Used to create prototype
Usage	obj.proto	Person.prototype

6. Inheritance using Prototype

```
JavaScript
function Animal(name) {
  this.name = name;
}

Animal.prototype.sound = function() {
  console.log("Animal sound");
};

function Dog(name) {
  Animal.call(this, name);
}

// Inheritance
Dog.prototype = Object.create(Animal.prototype);

const d = new Dog("Tommy");
d.sound();
```

7. Prototype Chain Lookup

```
JavaScript
const arr = [1,2,3];
```

 JS checks:

```
arr → Array.prototype → Object.prototype → null
```

👉 That's why:

```
JavaScript  
arr.push()
```

works (comes from `Array.prototype`)

8. Built-in Prototypes

- `Object.prototype`
- `Array.prototype`
- `String.prototype`

Example:

```
JavaScript  
Array.prototype.myMethod = function() {  
    console.log("Custom method");  
};  
  
[1,2].myMethod();
```

9. Important Rules

👉 JS follows **Prototype-based inheritance**, not classical OOP

👉 If property not found:
→ JS looks up in prototype chain

10. Common Mistakes

-  Confusing `prototype` with `__proto__`
-  Not using `Object.create()` properly
-  Overwriting prototype incorrectly

11. Interview Questions

Q1: What is prototype?

👉 Object from which other objects inherit properties

Q2: What is prototype chain?

👉 Chain of objects linked via `__proto__`

Q3: Difference between class and prototype?

👉 Class is syntactic sugar over prototype

Q4: Why use prototype?

👉 Memory optimization + inheritance

💡 12. Real-Life Analogy

👉 Prototype = **Parent**

```
Child object → Parent → Grandparent
```

If child doesn't have something → goes to parent

🚀 13. Advanced (Must Know)

Object.create()

```
JavaScript
const parent = {
  greet() {
    console.log("Hello");
  }
};

const child = Object.create(parent);
child.greet();
```

hasOwnProperty()

```
JavaScript
obj.hasOwnProperty("name");
```

👉 Check if property belongs to object (not prototype)

Final Interview Line



"JavaScript uses prototype-based inheritance where objects inherit properties from other objects through a prototype chain. Functions have a prototype property that is used when creating objects with `new`."



Next Step (Very Important for You)

Since you're preparing for **React + Backend + Interviews**, next learn:

-  Closures
-  Call, Apply, Bind
-  Event Loop
-  Async/Await Deep

If you want 

I can give you:

- ☒ 20 tricky prototype coding questions
- ☒ Visual diagrams (easy memory trick)
- ☒ Mini project using prototype (Bank system)

You said:

factory functions all concept

ChatGPT said:

Great  — **Factory Functions** are a very important concept in JavaScript OOP, especially for writing clean and reusable code.

Let's go **basic** → **advanced** → **interview level** 



1. What is a Factory Function?



A **factory function** is a function that **returns an object**

```
JavaScript
function createUser(name, age) {
  return {
    name: name,
    age: age
  };
}

const user1 = createUser("Aamir", 22);
```

👉 Instead of using **new**, we directly return objects

🔪 2. Factory Function vs Constructor Function

Feature	Factory Function	Constructor Function
Uses new	❌ No	✅ Yes
Simpler syntax	✅ Yes	❌ Slightly complex
this keyword	❌ Not required	✅ Required
Flexibility	✅ High	Medium

🧩 3. Adding Methods

```
JavaScript
function createUser(name) {
  return {
    name,
    greet() {
      console.log("Hello " + name);
    }
  };
}

const user = createUser("Aamir");
user.greet();
```

⚠️ 4. Problem (Memory Issue)

```
JavaScript
function createUser(name) {
  return {
    name,
    greet() {
      console.log("Hello");
    }
  };
}
```

👉 Every object creates **new copy of greet()** ❌

✅ 5. Solution (Using Shared Methods)

```
JavaScript
const userMethods = {
  greet() {
    console.log("Hello");
  }
};

function createUser(name) {
  const user = Object.create(userMethods);
  user.name = name;
  return user;
}

const u1 = createUser("Aamir");
u1.greet();
```

👉 Now method is shared 👍

🧬 6. Factory + Prototype Link

👉 `Object.create()` sets prototype

```
JavaScript
user.__proto__ === userMethods // true
```

🔒 7. Encapsulation (Private Variables)

👉 Factory functions support **closure-based privacy**


```
JavaScript
function createBankAccount(balance) {
  return {
    deposit(amount) {
      balance += amount;
    },
    getBalance() {
      return balance;
    }
  };
}

const acc = createBankAccount(1000);
console.log(acc.getBalance());
```

👉 **balance** is private 🔒

8. Real Example (Advanced)

```
JavaScript
function createCar(brand, speed) {
  return {
    brand,
    speed,
    drive() {
      console.log(brand + " is driving at " + speed);
    }
  };
}
```

9. Factory + Arrow Function

```
JavaScript
const createUser = (name) => ({
  name,
  greet() {
    console.log("Hello " + name);
  }
});
```

10. When to Use Factory Functions?

✅ When you want:

- Simpler syntax

- No **this** confusion
- Private variables (closures)
- More control over object creation

11. Common Mistakes

- ✗ Not using **Object.create()** for sharing
- ✗ Creating duplicate methods
- ✗ Confusing with constructor functions
- ✗ Ignoring closure benefits

12. Interview Questions

Q1: What is factory function?

- 👉 Function that returns an object

Q2: Factory vs Constructor?

- 👉 Factory doesn't use **new**, constructor does

Q3: How to create private variables?

- 👉 Using closures inside factory function

Q4: Memory optimization?

- 👉 Use shared object with **Object.create()**

13. Factory vs Class (Important)

Feature	Factory Function	Class
Private data	✅ Easy	✗ Hard (before #)
Simplicity	✅ High	Medium
Performance	Medium	Better with prototype

Final Interview Line



"Factory functions are functions that return objects and allow flexible object creation without using **new**. They can use closures for private variables and can be optimized using **Object.create** for shared methods."



Next Level (Recommended for You)

Since you're learning **React + Backend**, next learn:

- 👉 Closures (VERY IMPORTANT)
- 👉 Call, Apply, Bind
- 👉 ES6 Advanced (Destructuring, Spread)
- 👉 Design Patterns

If you want 🔥

I can give:

- ✅ 20 factory function practice questions
- ✅ Real interview coding problems
- ✅ Mini project (Bank / E-commerce using factory)

You said:

new operator class in js all concept

ChatGPT said:

Perfect 🔥 — this is **core JavaScript OOP + interview favorite**

Let's master **new operator + Classes in JavaScript** step-by-step 🚀



1. What is **new** Operator?

👉 **new** is used to **create an object from a constructor function or class**

```
JavaScript
function Person(name) {
  this.name = name;
}

const p1 = new Person("Aamir");
```



2. How **new** Works Internally (VERY IMPORTANT)

When you do:

```
JavaScript
const p1 = new Person("Aamir");
```

👉 JS does 4 steps:

◆ Step 1: Create empty object

```
JavaScript  
{}
```

◆ Step 2: Link prototype

```
JavaScript  
obj.__proto__ = Person.prototype
```

◆ Step 3: Bind **this**

```
JavaScript  
this → obj
```

◆ Step 4: Return object

```
JavaScript  
return obj
```

👉 This is **must-know for interviews**

3. Constructor Function

```
JavaScript  
function Car(brand) {  
    this.brand = brand;  
}  
  
const c1 = new Car("BMW");
```

👉 Naming convention:

```
JavaScript  
Car // capital letter
```



4. Classes in JavaScript (ES6)

👉 Classes are **syntactic sugar over prototype**

```
JavaScript
class Person {
  constructor(name) {
    this.name = name;
  }

  greet() {
    console.log("Hello " + this.name);
  }
}

const p1 = new Person("Aamir");
p1.greet();
```



5. Behind the Scenes (Class = Prototype)

```
JavaScript
Person.prototype.greet
```

👉 Methods are stored in **prototype**, not inside object



6. Constructor vs Class

Feature	Constructor Function	Class
Syntax	Old	Modern
Readability	Medium	High
Hoisting	Yes	No
Strict mode	No	Yes



7. Inheritance (Class)

```
JavaScript
class Animal {
  speak() {
    console.log("Animal sound");
  }
}

class Dog extends Animal {
  speak() {
    console.log("Dog barks");
  }
}

const d = new Dog();
d.speak();
```



8. super() Keyword

```
JavaScript
class Person {
  constructor(name) {
    this.name = name;
  }
}

class Student extends Person {
  constructor(name, marks) {
    super(name); // call parent constructor
    this.marks = marks;
  }
}
```

9. Private Properties (#)

```
JavaScript
class Bank {
  #balance = 0;

  deposit(amount) {
    this.#balance += amount;
  }

  getBalance() {
    return this.#balance;
  }
}
```

10. Static Methods

👉 Belong to class, not object

```
JavaScript
class MathUtil {
  static add(a, b) {
    return a + b;
  }
}

MathUtil.add(2, 3);
```

11. Common Mistakes

✗ Forgetting **new**

```
JavaScript
const p = Person("Aamir"); // wrong
```

✗ Not calling **super()** in child class

✗ Confusing class with real OOP (JS is prototype-based)

12. Interview Questions

Q1: What does **new do?**

👉 Creates object, links prototype, binds **this**, returns object

Q2: Class vs function?

👉 Class is syntactic sugar over prototype

Q3: Where are methods stored?

👉 In **prototype**

Q4: Why use **new**?

👉 To create instance of constructor/class

💡 13. Real-Life Analogy

👉 Class = Blueprint 🏗️

👉 Object = Actual house 🏠

🔥 14. Advanced Concept

Return value override

```
JavaScript
function Test() {
  this.name = "Aamir";
  return { name: "Ali" };
}

const t = new Test();
console.log(t.name); // Ali
```

👉 If constructor returns object → overrides

🎯 Final Interview Line



"The **new** operator creates an instance of a constructor or class by creating a new object, linking it to the prototype, binding **this**, and returning the object. Classes in JavaScript are syntactic sugar over prototype-based inheritance."



Next Step (Important for You)

You are building React + Backend → learn next:

👉 Closures (must)

👉 call, apply, bind

👉 Event loop

👉 Async JS deep

If you want 🔥

I can give:

- ✅ 30 OOP interview questions
- ✅ Real coding problems
- ✅ Mini project (Bank system using class + prototype)

You said:

explain both new operator and class in js all concept

ChatGPT said:

Alright — let's do a **complete, deep, interview-ready explanation** of both **new operator** and **class in JavaScript** in a **clear + connected way** 🚀

🔥 1. Big Picture

👉 In JavaScript:

- **new operator** → creates objects
- **class** → blueprint to create objects

👉 Together they form **modern OOP in JavaScript**

💡 2. **new** Operator (Full Concept)

✅ Definition

👉 **new** is used to **create an instance (object)** from a function or class

⚙️ Internal Working (MOST IMPORTANT)

```
JavaScript
function Person(name) {
  this.name = name;
}

const p1 = new Person("Aamir");
```

👉 Behind the scenes:

Step 1 Create empty object

```
JavaScript  
{}
```

Step 2 Link prototype

```
JavaScript  
obj.__proto__ = Person.prototype
```

Step 3 Bind **this**

```
JavaScript  
this = obj
```

Step 4 Return object

```
JavaScript  
return obj
```

Important Rule

👉 If constructor returns:

- **Primitive** → ignored
- **Object** → returned instead

```
JavaScript  
function Test() {  
  this.name = "Aamir";  
  return { name: "Ali" };  
}  
  
new Test().name; // Ali
```

Without **new**

```
JavaScript  
const p = Person("Aamir");
```

👉 ❌ **this** becomes **undefined (strict mode)** or **global object**

3. Constructor Function

👉 Old way before ES6

```
JavaScript
function Car(brand) {
  this.brand = brand;
}

const c1 = new Car("BMW");
```

👉 Naming convention:

- First letter **Capital**

4. Class in JavaScript (Full Concept)

Definition

👉 Class = **template / blueprint for creating objects**

Basic Syntax

```
JavaScript
class Person {
  constructor(name) {
    this.name = name;
  }

  greet() {
    console.log("Hello " + this.name);
  }
}

const p1 = new Person("Aamir");
```

Behind the Scenes

👉 Class internally works like:

```
JavaScript
Person.prototype.greet
```

👉 Means:

- Methods stored in **prototype**
- Not copied in every object

✂️ 5. Class vs Constructor Function

Feature	Constructor Function	Class
Syntax	Old	Modern
Readability	Medium	High
Hoisting	Yes	No
Strict Mode	No	Yes

🧬 6. Inheritance (Class)

```
JavaScript
class Animal {
  speak() {
    console.log("Animal sound");
  }
}

class Dog extends Animal {
  speak() {
    console.log("Dog barks");
  }
}

const d = new Dog();
d.speak();
```

🔄 7. super() Keyword

👉 Used to call parent constructor

```

JavaScript
class Person {
  constructor(name) {
    this.name = name;
  }
}

class Student extends Person {
  constructor(name, marks) {
    super(name);
    this.marks = marks;
  }
}

```

8. Private Properties

```

JavaScript
class Bank {
  #balance = 0;

  deposit(amount) {
    this.#balance += amount;
  }

  getBalance() {
    return this.#balance;
  }
}

```

9. Static Methods

👉 Belong to class, not instance

```

JavaScript
class MathUtil {
  static add(a, b) {
    return a + b;
  }
}

MathUtil.add(2, 3);

```

10. Relationship: **new** + Class

👉 Class cannot work without **new**

```
JavaScript
const p = new Person("Aamir");
```

👉 If you do:

```
JavaScript
Person("Aamir"); // ❌ Error
```

🧠 11. Prototype Connection

👉 When using class:

```
JavaScript
p1.__proto__ === Person.prototype // true
```

👉 Chain:

```
p1 → Person.prototype → Object.prototype → null
```

⚠️ 12. Common Mistakes

- ❌ Forgetting **new**
- ❌ Not calling **super()**
- ❌ Confusing class with real OOP
- ❌ Thinking methods stored in object

🧪 13. Interview Questions

Q1: What does **new** do?

👉 Creates object, links prototype, binds **this**, returns object

Q2: What is class?

👉 Blueprint for creating objects

Q3: Class vs prototype?

👉 Class is syntactic sugar over prototype

Q4: Where methods stored?

👉 In prototype

💡 14. Real-Life Analogy

- 👉 Class = **Blueprint** 🏗️
- 👉 new = **Builder** 🧑
- 👉 Object = **House** 🏠

🎯 Final Interview Answer



"In JavaScript, the **new** operator is used to create instances of constructor functions or classes by creating a new object, linking it to the prototype, binding **this**, and returning it. Classes are syntactic sugar over prototype-based inheritance and provide a cleaner way to implement OOP concepts like inheritance, encapsulation, and polymorphism."



Next Step (VERY IMPORTANT FOR YOU)

You're doing **React + Backend + Interviews**, so next learn:

- 👉 Closures (VERY IMPORTANT)
- 👉 call, apply, bind
- 👉 Event Loop
- 👉 Async/Await deep

If you want 🔥

I can give:

- ✅ 30 interview questions on OOP
- ✅ Coding problems
- ✅ Mini project (Bank system using class + new)

You said:

inheritance all concept

ChatGPT said:

Perfect 🔥 — **Inheritance** is one of the **most important OOP concepts** in JavaScript (very common in interviews + real projects).

Let's go **basic** → **advanced** → **interview level** 🚀

1. What is Inheritance?

👉 Inheritance means:

One object/class can use properties & methods of another

📌 Goal:

- Code reuse
- Avoid duplication

Real-Life Example

👉 Parent: **Animal**

👉 Child: **Dog**

Dog can:

- Use Animal's properties
- Add its own features

2. Inheritance using Classes (ES6)

Basic Example

```
JavaScript
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(this.name + " makes sound");
  }
}

class Dog extends Animal {
  bark() {
    console.log(this.name + " barks");
  }
}

const d = new Dog("Tommy");
d.speak(); // inherited
d.bark();  // own
```


3. **extends** Keyword

👉 Used to create inheritance

```
JavaScript
class Child extends Parent
```

4. **super()** Keyword (VERY IMPORTANT)

👉 Calls parent constructor

```
JavaScript
class Student extends Animal {
  constructor(name, marks) {
    super(name); // parent constructor call
    this.marks = marks;
  }
}
```

⚠️ **Rule:**

👉 Must call **super()** before using **this**

5. **Method Overriding (Polymorphism)**

```
JavaScript
class Dog extends Animal {
  speak() {
    console.log(this.name + " barks");
  }
}
```

👉 Child overrides parent method



6. Inheritance using Prototype (Old Way)

```

JavaScript
function Animal(name) {
    this.name = name;
}

Animal.prototype.speak = function() {
    console.log(this.name + " makes sound");
};

function Dog(name) {
    Animal.call(this, name);
}

// link prototype
Dog.prototype = Object.create(Animal.prototype);

const d = new Dog("Tommy");
d.speak();

```



7. Prototype Chain

```
d → Dog.prototype → Animal.prototype → Object.prototype → null
```



JS searches properties in this chain



8. Types of Inheritance (JS Perspective)

1. Single Inheritance

```
Animal → Dog
```

2. Multilevel

```
Animal → Dog → Puppy
```

3. Hierarchical

```

Animal → Dog
        → Cat

```

👉 JS does NOT support multiple inheritance (but can use mixins)

🧪 9. Mixins (Advanced)

👉 Alternative to multiple inheritance

```
JavaScript
const canEat = {
  eat() {
    console.log("Eating...");
  }
};

const canWalk = {
  walk() {
    console.log("Walking...");
  }
};

const person = Object.assign({}, canEat, canWalk);
person.eat();
person.walk();
```

🔒 10. Private + Inheritance

```
JavaScript
class Parent {
  #secret = "hidden";

  show() {
    console.log(this.#secret);
  }
}
```

👉 ❌ Child cannot access `#secret` directly

⚠️ 11. Common Mistakes

- ❌ Forgetting `super()`
- ❌ Using `this` before `super()`
- ❌ Breaking prototype chain
- ❌ Confusing inheritance with copying

12. Interview Questions

Q1: What is inheritance?

👉 Mechanism where one class acquires properties of another

Q2: How is inheritance implemented in JS?

👉 Using prototype chain or **extends** keyword

Q3: What is **super()**?

👉 Calls parent constructor

Q4: Does JS support multiple inheritance?

👉 ❌ No (use mixins)

13. Real-Life Analogy

👉 Parent = Human 🧑

👉 Child = Student 🎓

Student inherits:

- name
- behavior

Final Interview Answer



"Inheritance in JavaScript allows a class or object to acquire properties and methods from another using prototype chaining or the **extends** keyword. It promotes code reuse and supports method overriding using polymorphism."

14. Next Step (Very Important)

To master JS fully:

- 👉 Prototypes (already done ✅)
- 👉 Closures (VERY IMPORTANT)
- 👉 call, apply, bind
- 👉 Event Loop

If you want 🔥

I can give:

- ✅ 20 inheritance coding questions
- ✅ Real interview problems
- ✅ Mini project (Bank / E-commerce using inheritance)

You said:

Object Oriented Programming structure our code • prototypes • New Operator
• constructors • classes • keywords (extends, super)

2/2

ChatGPT said:

Perfect 🔥 — this is a **complete OOP revision sheet for JavaScript** (exactly what interviewers expect). I'll explain each point **clearly + connected** so you see the full picture 🚀

1. Object-Oriented Programming (OOP)

👉 OOP = **way to structure code using objects**

Goal:

- Reusable code
- Clean structure
- Easy maintenance

4 Pillars:

- Encapsulation 🗝️
- Inheritance 🧬
- Polymorphism 🔄
- Abstraction 🎭

2. Structure Our Code (Why OOP?)

❌ Without OOP:

```
JavaScript
let name = "Aamir";
let age = 22;
```

✅ With OOP:

```
JavaScript
const user = {
  name: "Aamir",
  age: 22,
  greet() {
    console.log("Hello");
  }
};
```

👉 Code becomes:

- Organized
- Modular
- Scalable

3. Prototypes (Core of JS OOP)

👉 JavaScript is **prototype-based**

📌 **Concept:**

```
JavaScript
function Person(name) {
  this.name = name;
}

Person.prototype.greet = function() {
  console.log("Hello " + this.name);
};
```

- 👉 All objects share **greet()**
- 👉 Saves memory

Prototype Chain:

```
object → prototype → Object.prototype → null
```

4. New Operator

👉 Used to create object from constructor/class

```
JavaScript  
const p1 = new Person("Aamir");
```

Internally:

1. Create object
2. Link prototype
3. Bind **this**
4. Return object

5. Constructors

👉 Special function to create objects

```
JavaScript  
function Car(brand) {  
    this.brand = brand;  
}  
  
const c1 = new Car("BMW");
```

Rules:

- Capital letter (Car)
- Use **new**

6. Classes (Modern JS)

👉 Cleaner way to write constructors + prototypes

```
JavaScript
class Person {
  constructor(name) {
    this.name = name;
  }

  greet() {
    console.log("Hello " + this.name);
  }
}

const p1 = new Person("Aamir");
```

👉 Behind the scenes → **prototype**

7. Keywords

◆ **extends (Inheritance)**

```
JavaScript
class Animal {
  speak() {
    console.log("Animal sound");
  }
}

class Dog extends Animal {
  bark() {
    console.log("Dog barks");
  }
}
```

👉 Dog inherits Animal

◆ **super (Parent Access)**

```
JavaScript
class Animal {
  constructor(name) {
    this.name = name;
  }
}

class Dog extends Animal {
  constructor(name) {
    super(name); // call parent
  }
}
```

⚠ **Rule:**

👉 Must call **super()** before using **this**

🔄 **8. How Everything Connects (IMPORTANT)**

👉 Flow:

```
Constructor / Class
  ↓
new Operator
  ↓
Object Created
  ↓
Prototype Linked
  ↓
Inheritance (extends)
```

🧠 **9. Memory Optimization**

👉 Methods stored in:

```
JavaScript
Person.prototype
```

👉 Not inside every object

10. Common Mistakes

- ✗ Forgetting `new`
- ✗ Not calling `super()`
- ✗ Confusing class with real OOP
- ✗ Ignoring prototype

11. Interview Questions

Q1: What is prototype?

- 👉 Object used for inheritance

Q2: What does `new` do?

- 👉 Creates object + links prototype

Q3: Class vs constructor?

- 👉 Class = syntactic sugar

Q4: What is `extends`?

- 👉 Used for inheritance

Q5: What is `super()`?

- 👉 Calls parent constructor

12. Real-Life Analogy

- 👉 Class = Blueprint 
- 👉 `new` = Builder 
- 👉 Object = House 
- 👉 Prototype = Shared features

Final Interview Answer

- 👉
"JavaScript uses prototype-based OOP where objects inherit from other objects. Constructors and classes are used to create objects using the `new` operator. Classes provide a cleaner syntax over prototypes, and inheritance is implemented using `extends` and `super`."

Next Step (Very Important for You)

You are preparing for **React + Backend**, so next:

- 👉 Closures (VERY IMPORTANT)
- 👉 call, apply, bind
- 👉 Event Loop
- 👉 Async JS

If you want 🔥

I can give:

- ✅ Complete OOP revision PDF
- ✅ 30 interview coding questions
- ✅ Mini project (Bank system using all OOP concepts)

You said:

11. Interview Questions Q1: What is prototype? 👉 Object used for inheritance
 Q2: What does new do? 👉 Creates object + links prototype Q3: Class vs constructor? 👉 Class = syntactic sugar Q4: What is extends? 👉 Used for inheritance Q5: What is super()? 👉 Calls parent constructor 💡 12. Real-Life Analogy 👉 Class = Blueprint 🏗️ 👉 new = Builder 👤 👉 Object = House 🏠
 👉 Prototype = Shared features 🎯 Final Interview Answer 👉 "JavaScript uses prototype-based OOP where objects inherit from other objects. Constructors and classes are used to create objects using the new operator. Classes provide a cleaner syntax over prototypes, and inheritance is implemented using extends and super." 🚀 Next Step (Very Important for You) You are preparing for React + Backend, so next: 👉 Closures (VERY IMPORTANT) 👉 call, apply, bind 👉 Event Loop 👉 Async JS If you want 🔥 I can give: ✅ Complete OOP revision PDF ✅ 30 interview coding questions ✅ Mini project (Bank system using all OOP concepts) complete solution

ChatGPT said:

Perfect 🔥 — let's build a **complete OOP mini project: Bank System (with full solution)** using:

- 👉 Classes
- 👉 Inheritance
- 👉 Encapsulation
- 👉 new, extends, super
- 👉 Real interview-level structure



Bank System Project (Full Code)



Features:

- Create Account
- Deposit Money
- Withdraw Money
- Check Balance
- Savings Account (interest feature)



1. Base Class (BankAccount)

```
JavaScript
class BankAccount {
  #balance = 0; // private property

  constructor(name, accountNumber) {
    this.name = name;
    this.accountNumber = accountNumber;
  }

  deposit(amount) {
    if (amount > 0) {
      this.#balance += amount;
      console.log(`${amount} deposited`);
    } else {
      console.log("Invalid amount");
    }
  }

  withdraw(amount) {
    if (amount <= this.#balance) {
      this.#balance -= amount;
      console.log(`${amount} withdrawn`);
    } else {
      console.log("Insufficient balance");
    }
  }

  getBalance() {
    return this.#balance;
  }
}
```

2. Child Class (SavingsAccount)

Using **inheritance** (**extends**)

```
JavaScript
class SavingsAccount extends BankAccount {
  constructor(name, accountNumber, interestRate) {
    super(name, accountNumber); // call parent
    this.interestRate = interestRate;
  }

  addInterest() {
    let interest = this.getBalance() * this.interestRate /
100;
    this.deposit(interest);
    console.log(`Interest added: ${interest}`);
  }
}
```

3. Using the System

```
JavaScript
const acc1 = new BankAccount("Aamir", 101);

acc1.deposit(1000);
acc1.withdraw(300);

console.log("Balance:", acc1.getBalance());
```

4. Using Inheritance Class

```
JavaScript
const savAcc = new SavingsAccount("Ali", 102, 10);

savAcc.deposit(2000);
savAcc.addInterest();

console.log("Final Balance:", savAcc.getBalance());
```

5. Concepts Covered

Concept	Where Used
Class	BankAccount, SavingsAccount

Concept	Where Used
new operator	Creating objects
Constructor	constructor()
Encapsulation	#balance
Inheritance	extends
super()	Parent constructor
Methods	deposit, withdraw

6. Output Example

```
1000 deposited
300 withdrawn
Balance: 700

2000 deposited
Interest added: 200
Final Balance: 2200
```

7. Interview Questions from This Project

Q1: Why use #balance?

👉 To make it **private (encapsulation)**

Q2: Why super()?

👉 To call parent constructor

Q3: Why inheritance?

👉 To reuse code (deposit, withdraw)

Q4: Where are methods stored?

👉 In **prototype**

8. Upgrade Ideas (Very Important)

 You can improve project:

- Add **Login system**
- Add **Transaction history**
- Add **Current Account**
- Connect with **Database (MongoDB/MySQL)**
- Build **Frontend in React**

9. Practice Tasks (Try Yourself)

1. Add transfer money method
2. Add minimum balance rule
3. Add account type (saving/current)
4. Add user authentication

Final Interview Line



"This project demonstrates OOP in JavaScript using classes, inheritance, encapsulation, and the new operator. It shows how real-world systems like banking can be modeled using reusable and scalable code."

Next Step (Your Roadmap)

Now you should learn:

-  Closures (VERY IMPORTANT )
-  call, apply, bind
-  Event Loop
-  Async/Await

If you want next level 

I can give you:

- ✓ 30 real interview coding questions (TCS, Infosys, Amazon)
- ✓ Full backend API for this bank system
- ✓ React frontend for this project

Just tell me 

You said:

Give me 30 real interview coding questions (TCS, Infosys, Amazon)

ChatGPT said:

Good — this is exactly how you prepare smartly for placements 🔥
I'll give you **30 REAL interview-level coding questions** (TCS, Infosys, Amazon level)

👉 arranged from **easy** → **medium** → **hard**

 EASY LEVEL (TCS / Infosys Basics)

1. Check even or odd

2. Find largest of 3 numbers

3. Reverse a number

4. Check palindrome (number/string)

5. Count vowels in string

6. Factorial of number

7. Fibonacci series (n terms)

8. Sum of digits

9. Check prime number

10. Swap two numbers (without temp)

 MEDIUM LEVEL (Infosys / Wipro / Capgemini)

11. Remove duplicates from array

JavaScript

Input: [1,2,2,3,4,4] → Output: [1,2,3,4]

12. Find second largest number

```
JavaScript  
Input: [10,5,20,8] → Output: 10
```

13. Check anagram

```
JavaScript  
"listen" → "silent"
```

14. Find missing number in array

```
JavaScript  
[1,2,4,5] → Missing = 3
```

15. Count frequency of elements

```
JavaScript  
[1,2,2,3] → {1:1,2:2,3:1}
```

16. Reverse words in sentence

```
JavaScript  
"I love JS" → "JS love I"
```

17. Find intersection of two arrays

```
JavaScript  
[1,2,3] & [2,3,4] → [2,3]
```

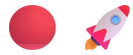
18. Flatten array

```
JavaScript  
[1,[2,[3]]] → [1,2,3]
```

19. First non-repeating character

```
JavaScript  
"aabbccde" → c
```

20. Rotate array by k steps



HARD LEVEL (Amazon / Product-based)

21. Two Sum Problem

```
JavaScript  
nums = [2,7,11,15], target = 9 → [0,1]
```

22. Longest substring without repeating characters

23. Valid parentheses

```
JavaScript  
"() [] {}" → true
```

24. Merge two sorted arrays

25. Kadane's Algorithm (Max subarray sum)

26. Group anagrams

```
JavaScript  
["eat","tea","tan"] → [ ["eat","tea"], ["tan"] ]
```

27. Implement debounce function (VERY IMPORTANT JS)

28. Deep clone object

29. LRU Cache implementation

30. Promise.all implementation (Advanced JS)



BONUS (VERY IMPORTANT FOR YOU)

Since you're doing **React + Backend**, these are MUST:

- 👉 Array + String problems (1–20)
- 👉 JS Core (27, 28, 30)
- 👉 DSA (21–26, 29)



How to Practice (Smart Strategy)

👉 Day-wise:

- Day 1–3 → Easy (1–10)
- Day 4–7 → Medium (11–20)
- Day 8–15 → Hard (21–30)



Interview Tip

👉 Don't just solve — be ready to explain:

- Time complexity
- Space complexity
- Edge cases



Next Step

If you want serious placement prep 🔥

I can give you:

- ✅ All 30 questions with **full solutions + explanation**
- ✅ Mock interview questions
- ✅ Company-wise preparation (TCS, Infosys, Amazon)
- ✅ DSA roadmap (beginner → advanced)

Just tell me: **“give solutions”** 👉

You said:

Give me 30 real interview coding questions (TCS, Infosys, Amazon) with solution

ChatGPT said:

Great 🧯 — now we move to **real preparation mode**.

Here are **30 interview coding questions with clean solutions (JavaScript)** from **TCS / Infosys / Amazon level**.

I'll keep solutions **simple + interview-friendly** (with logic clarity).

 EASY (1–10)

1. Even or Odd

```
JavaScript
function isEven(n) {
    return n % 2 === 0;
}
```

2. Largest of 3 numbers

```
JavaScript
function largest(a, b, c) {
    return Math.max(a, b, c);
}
```

3. Reverse a number

```
JavaScript
function reverseNum(n) {
    return Number(n.toString().split("").reverse().join(""));
}
```

4. Palindrome

```
JavaScript
function isPalindrome(str) {
    return str === str.split("").reverse().join("");
}
```

5. Count vowels

```
JavaScript
function countVowels(str) {
    return str.match(/[aeiou]/gi)?.length || 0;
}
```

6. Factorial

```
JavaScript
function factorial(n) {
    let res = 1;
    for (let i = 1; i <= n; i++) res *= i;
    return res;
}
```

7. Fibonacci

```
JavaScript
function fibonacci(n) {
    let a = 0, b = 1;
    for (let i = 0; i < n; i++) {
        console.log(a);
        [a, b] = [b, a + b];
    }
}
```

8. Sum of digits

```
JavaScript
function sumDigits(n) {
    return n.toString().split("").reduce((a, b) => a + +b, 0);
}
```

9. Prime number

```
JavaScript
function isPrime(n) {
  if (n < 2) return false;
  for (let i = 2; i <= Math.sqrt(n); i++) {
    if (n % i === 0) return false;
  }
  return true;
}
```

10. Swap without temp

```
JavaScript
let a = 5, b = 10;
[a, b] = [b, a];
```

MEDIUM (11–20)

11. Remove duplicates

```
JavaScript
const unique = arr => [...new Set(arr)];
```

12. Second largest

```
JavaScript
function secondLargest(arr) {
  arr = [...new Set(arr)].sort((a,b)=>b-a);
  return arr[1];
}
```

13. Anagram

```
JavaScript
function isAnagram(a, b) {
  return a.split('').sort().join('') ===
b.split('').sort().join('');
}
```

14. Missing number

```
JavaScript
function missing(arr, n) {
  let sum = n*(n+1)/2;
  return sum - arr.reduce((a,b)=>a+b,0);
}
```

15. Frequency count

```
JavaScript
function freq(arr) {
  return arr.reduce((acc,x)=>{
    acc[x] = (acc[x]||0)+1;
    return acc;
  },{});
}
```

16. Reverse words

```
JavaScript
function reverseWords(str) {
  return str.split(" ").reverse().join(" ");
}
```

17. Intersection

```
JavaScript
function intersection(a, b) {
  return a.filter(x => b.includes(x));
}
```

18. Flatten array

```
JavaScript
function flatten(arr) {
  return arr.flat(Infinity);
}
```

19. First non-repeating char

```
JavaScript
function firstUnique(str) {
  for (let c of str) {
    if (str.indexOf(c) === str.lastIndexOf(c)) return c;
  }
}
```

20. Rotate array

```
JavaScript
function rotate(arr, k) {
  k = k % arr.length;
  return [...arr.slice(-k), ...arr.slice(0, -k)];
}
```



HARD (21–30)

21. Two Sum

```
JavaScript
function twoSum(nums, target) {
  let map = {};
  for (let i = 0; i < nums.length; i++) {
    let diff = target - nums[i];
    if (map[diff] !== undefined) return [map[diff], i];
    map[nums[i]] = i;
  }
}
```


22. Longest substring (no repeat)

```
JavaScript
function longestSubstr(s) {
    let set = new Set(), l = 0, max = 0;
    for (let r = 0; r < s.length; r++) {
        while (set.has(s[r])) {
            set.delete(s[l++]);
        }
        set.add(s[r]);
        max = Math.max(max, r - l + 1);
    }
    return max;
}
```

23. Valid parentheses

```
JavaScript
function valid(s) {
    let stack = [], map = {'(': ')', '[': ']', '{': '}'};
    for (let c of s) {
        if (map[c]) {
            if (stack.pop() !== map[c]) return false;
        } else stack.push(c);
    }
    return stack.length === 0;
}
```

24. Merge sorted arrays

```
JavaScript
function merge(a,b) {
    return [...a, ...b].sort((x,y)=>x-y);
}
```

25. Kadane's Algorithm

```
JavaScript
function maxSubarray(arr) {
    let max = arr[0], sum = arr[0];
    for (let i = 1; i < arr.length; i++) {
        sum = Math.max(arr[i], sum + arr[i]);
        max = Math.max(max, sum);
    }
    return max;
}
```

26. Group Anagrams

```
JavaScript
function groupAnagrams(arr) {
    let map = {};
    for (let word of arr) {
        let key = word.split('').sort().join("");
        if (!map[key]) map[key] = [];
        map[key].push(word);
    }
    return Object.values(map);
}
```

27. Debounce

```
JavaScript
function debounce(fn, delay) {
    let timer;
    return function(...args) {
        clearTimeout(timer);
        timer = setTimeout(()=>fn(...args), delay);
    };
}
```

28. Deep Clone

```
JavaScript
function deepClone(obj) {
    return JSON.parse(JSON.stringify(obj));
}
```

29. LRU Cache (Basic)

```
JavaScript
class LRU {
  constructor(limit) {
    this.cache = new Map();
    this.limit = limit;
  }

  get(key) {
    if (!this.cache.has(key)) return -1;
    let val = this.cache.get(key);
    this.cache.delete(key);
    this.cache.set(key, val);
    return val;
  }

  put(key, val) {
    if (this.cache.has(key)) this.cache.delete(key);
    else if (this.cache.size === this.limit) {
      this.cache.delete(this.cache.keys().next().value);
    }
    this.cache.set(key, val);
  }
}
```

30. Promise.all

```
JavaScript
function myPromiseAll(promises) {
  return new Promise((resolve, reject) => {
    let res = [], count = 0;

    promises.forEach((p, i) => {
      Promise.resolve(p).then(val => {
        res[i] = val;
        count++;
        if (count === promises.length) resolve(res);
      }).catch(reject);
    });
  });
}
```

Final Advice (Important)

👉 For **TCS / Infosys**:

- Focus on **1–20**

👉 For **Amazon / Product-based**:

- Focus on **21–30**

If You Want Next Level

I can help you with:

- ✓ Time & Space Complexity for each
- ✓ Mock interview (real questions)
- ✓ DSA roadmap (placement ready)
- ✓ React + Backend interview prep

Just say: “**mock interview**” or “**roadmap**” 👍

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).